

完整课程 · 第一版

普通人友好

不要求会编程

普通人 也能看懂的 GitHub

不背命令，只讲清楚：AI Agent 为什么总要用它，以及你怎样让它更安全地干活。

约 60—90 分钟

可以分几次随手看

12 个短章

每章只回答一个问题

自然语言为主

命令交给 Agent 执行

这门课不教你成为程序员。

它只希望你以后打开 GitHub 不再发懵，听见 Repo、Clone、Commit、Branch 时知道大概发生了什么；当 Agent 准备下载项目、修改文件或上传 GitHub 时，你也知道应该先问哪一句。

课程目录

- | | |
|--------------------------------|-----------------------------|
| 01 为什么用了 Agent，就会碰到 GitHub | 02 Git、GitHub、Repo 到底什么关系 |
| 03 打开一个 GitHub 项目，先看哪里 | 04 Download、Clone、Fork 怎么选 |
| 05 第一次让 Agent 配置 Git | 06 Status、Diff、Commit 在记录什么 |
| 07 Branch 和 Pull Request 为什么有用 | 08 Push、Pull 与本地、云端同步 |
| 09 给 Agent 留一份项目说明书 | 10 一套普通人可用的 Agent 工作流 |
| 11 公开仓库、API Key 与安全边界 | 12 五个常见场景，应该怎样开口 |

GitHub 没有看起来那么神秘

01

这一章只回答：为什么普通人突然也要懂一点 GitHub？

为什么用了 Agent，就会碰到 GitHub

你只是让 Agent 帮你安装一个 Skill，它却说：“我先 clone 这个 repository，检查 README，再看看当前 branch。”

很多普通人第一次接触 GitHub，不是因为想学编程，而是因为 Agent 把它带到了面前。

以前，写程序的人需要自己找项目、下载代码、保存版本、处理协作。现在 Agent 能代替我们完成不少操作，但它仍然要在这些项目和版本上工作。于是，过去属于程序员的基础设施，开始出现在普通人的使用场景里。

你可能会在这些时候碰到 GitHub：

- 让 Agent 安装一个开源工具、插件、Skill 或 MCP；
- 让 Agent 接手别人发布的网页、应用或自动化项目；
- 把自己做的 HTML、脚本或资料保存起来；
- 让 Codex、Claude Code 等 Agent 长期修改同一个项目；
- 想把自己的成果分享给别人，或者换一台电脑继续工作。

但这并不意味着“没有 GitHub，Agent 就不能工作”。本地 Agent 完全可以直接操作普通文件夹。Git 和 GitHub 的价值，是让这些操作多出记录、退路、同步和协作。

你要学的不是“怎么替 Agent 敲命令”，而是怎么听懂它准备做什么。

Agent 把普通人带进了项目世界；懂一点 GitHub，是为了更清楚地指挥和验收它。

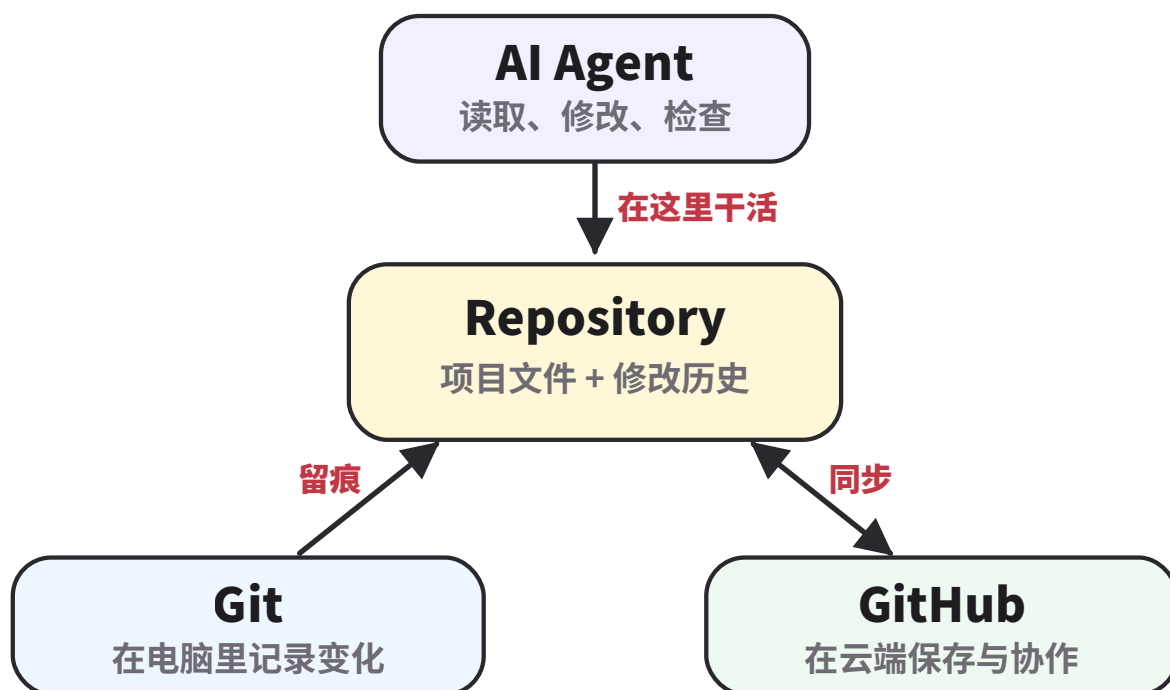
02

这一章只回答：三个最容易混淆的词有什么区别？

Git、GitHub、Repo 到底什么关系

先把三个概念摆到一张图里：

一张图看懂：Agent 在项目里动手，Git 负责留痕，GitHub 负责把项目放到云端



Repo 是大家共同围绕的“项目本体”；Git 和 GitHub 不是一回事，也都不等于 Agent。

Git：电脑里的项目存档系统

Git 是一种版本控制系统。它会记录项目在不同时刻的状态，让人知道改了什么、谁改的、什么时候改，以及必要时能否回到以前。

它可以完全在本地工作。没有网络，你照样可以让 Git 保存一次版本、比较前后差异。

GitHub：网上的项目仓库与协作平台

GitHub 用来托管 Git 仓库。除了把项目放到网上，它还提供 Issues、Pull Requests、代码审查、权限管理等功能。

如果把 Git 比作电脑里的“项目时光机”，GitHub 更像存放这些时光机档案的云端项目大厅。

Repository：带历史记录的项目文件夹

Repository 经常简称 Repo，中文一般翻译成“仓库”。它包含项目的文件、文件夹以及修改历史，可以是公开的，也可以是私有的。

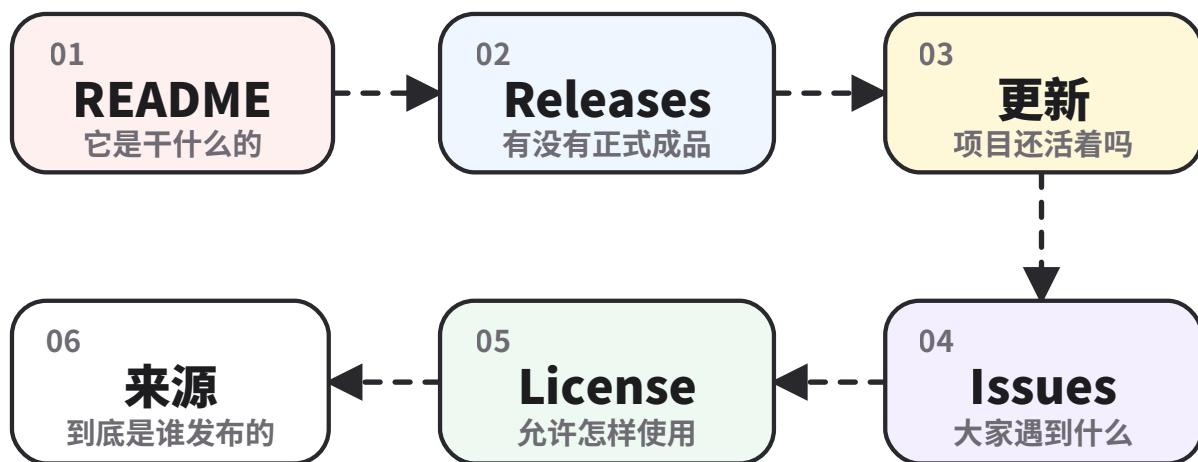


最短记法
Git 管本地历史，GitHub 管云端项目，Repo 是它们共同围绕的项目单位。

打开一个 GitHub 项目，先看哪里

GitHub 项目页面看起来信息很多，但普通人不用每个按钮都懂。先看下面六个地方，通常已经能判断出大概。

第一次打开陌生仓库，照着这条路线看



不用逛完整个页面，先走完这六站

1 README：这个项目到底是干什么的

一般会说明用途、安装方式、使用方法、适用系统和常见问题。README 写得含糊，后面往往更容易踩坑。

2 Releases：有没有面向使用者的正式版本

作者可以在这里发布可下载的版本、安装包和更新说明。对小白来说，Release 常常比源代码压缩包更接近“可以直接使用的成品”。

3 最近更新时间：项目还活着吗

几年没更新不代表一定不能用，但如果它依赖新系统、新模型或新接口，过旧就可能成为实际风险。

4

Issues：大家正在遇到什么问题

Issues 可以记录 Bug、建议和任务。问题多不一定说明项目差，关键看维护者是否回应、严重问题是否长期无人处理。

5

License：别人允许你怎样使用

公开可见不等于随便复制、商用和二次发布。没有许可证时，默认版权规则仍然存在。

6

来源：它是谁发布的

优先找作者、公司、学校或官方组织的仓库。名字很像、搜索排名很靠前，也可能只是第三方复制品。

Star 不是安全检测报告

Star 更像点赞或收藏，可以说明关注度，不能证明每个人都检查过代码。一个项目很火，也不代表你可以闭着眼睛运行它给出的安装命令。

还有一个很容易忽视的区别：GitHub 是代码和项目平台，不是经过统一审核的软件商店。仓库里的说明、命令和文件由项目维护者负责；把陌生命令交给 Agent 执行前，最好先让它解释影响范围。

可以直接发给 Agent

先不要安装。请阅读这个项目的 README、Releases、最近更新、主要 Issues 和 License，告诉我它来自谁、是否仍在维护、适不适合我的系统，以及安装过程会下载和修改什么。

最小阅读顺序

README → Releases → 最近更新 → Issues → License → 发布者身份。

04

这一章只回答：几个“复制项目”的按钮有什么不同？

Download、Clone、Fork 怎么选

Download ZIP

下载项目当前文件的压缩包。简单直接，但通常没有完整版本历史，也不会自动保持与原仓库的同步关系。

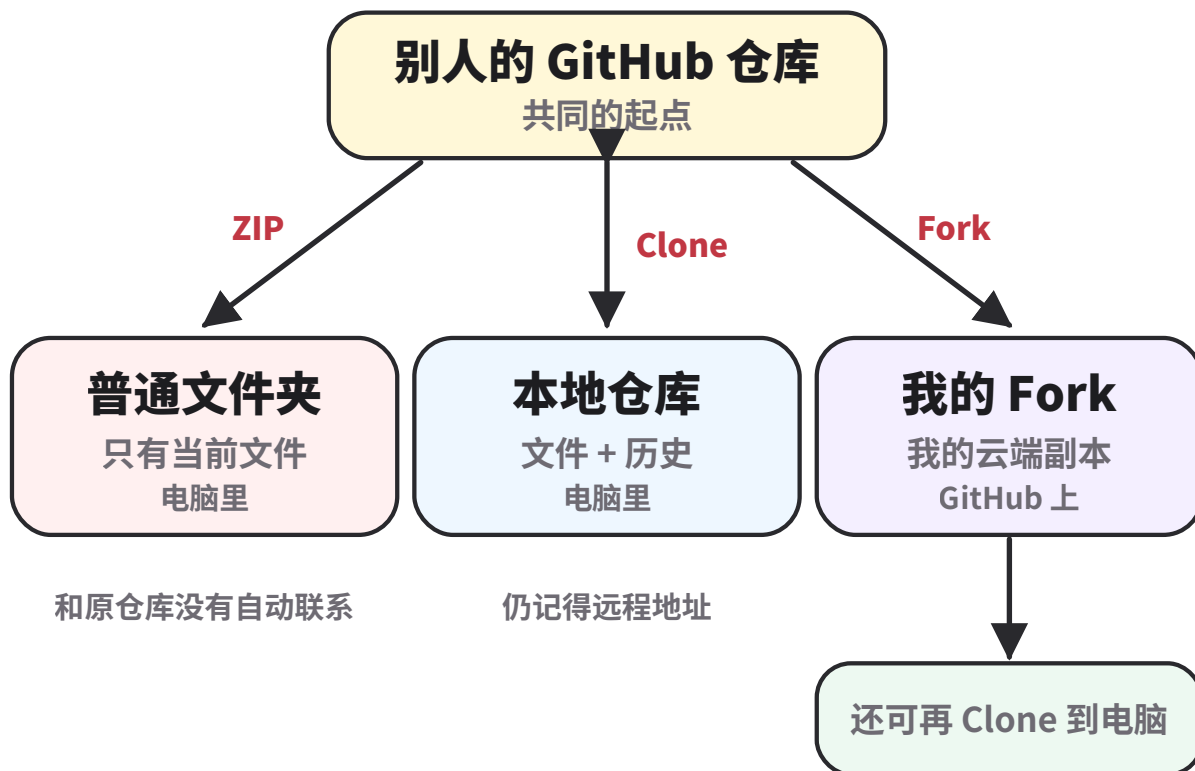
Clone

把仓库文件、历史和远程地址一起复制到电脑。适合准备长期修改、获取后续更新或让 Agent 正式接手的项目。

这两个动作看起来都像“下载”，区别在于你拿回来的是否仍然是一个完整 Git 仓库。

下载 ZIP 像把当前项目复印一份；Clone 像把整套档案和以后联系原仓库的方式一起带回来。

三个动作，复制的“位置”和“关系”不一样



那 Fork 又是什么？

Fork 是先在 GitHub 上把别人的仓库复制到你自己的账号名下。你可以独立修改自己的版本，同时保留它来自哪个上游项目的关系。

它常用于参与开源项目，也可以用来保留自己的长期分支。不过，如果你只是想在电脑上试用工具，通常不用一上来就 Fork。

只想看看或临时使用

优先找 Releases；没有成品时，再考虑 Download ZIP。

准备让 Agent 长期修改

Clone 更合适，因为历史和远程关系都在。

想基于别人项目做自己的版本

先 Fork 到自己的 GitHub，再 Clone 到电脑。

只是收藏以后再看

点 Star 即可，不等于下载或复制项目。

一句话带走

ZIP 是文件快照，Clone 是可继续工作的完整副本，Fork 是放到自己 GitHub 名下的项目副本。

不背命令，也能让 Agent 正确使用 Git

05

这一章只回答：第一次使用时，到底要配置什么？

第一次让 Agent 配置 Git

很多教程会从一排命令开始，普通人看到这里就想退出。其实第一次配置 Git，核心只有三件事。

1

电脑里有没有 Git

Agent 可以先运行版本检查。没有的话，再从 Git 官方渠道或适合当前系统的可信方式安装。

2

提交记录写谁的名字和邮箱

Git 会把作者身份写进新的 Commit。它是提交身份，不是让 Git 读取你的邮箱密码。

3

怎样连接 GitHub 账号

优先使用 GitHub Desktop 或 GitHub CLI 的浏览器授权。不要把 Token、密码直接发进 Agent 对话。

Agent 可能会运行下面这些命令。你不用记，只需要知道它们在检查或设置什么：

```
# 检查 Git 是否存在
git --version

# 设置以后提交时显示的作者信息
git config --global user.name "你的名字"
git config --global user.email "你的提交邮箱"

# 通过 GitHub CLI 发起登录，通常可走浏览器确认
gh auth login
```

全局配置意味着这台电脑上的所有仓库默认使用同一身份；如果电脑多人共用，或者工作与个人项目要分开，可以只为当前仓库配置。

如果不想公开真实邮箱，可以在 GitHub 邮箱设置里使用其提供的 no-reply 提交地址。关键不是死记格式，而是让 Agent 根据你的账号设置核对。

GitHub 登录已经不是“输入账号密码”那么简单

GitHub 的 Git 操作不再支持普通账户密码认证。对小白来说，GitHub Desktop 或 GitHub CLI 的浏览器登录通常比自己创建和管理 Token 更省心。

可以直接发给 Agent

请检查这台电脑是否已经安装 Git 和 GitHub CLI。先只报告状态，不要修改。然后说明当前 Git 用户名、邮箱配置在全局还是当前仓库；如果需要登录 GitHub，优先使用浏览器授权，不要让我把密码或 Token 发到对话里。

一句话带走

第一次配置就是：确认 Git、设置提交身份、用安全方式连接 GitHub。

06

这一章只回答：Agent 改文件后，你怎样知道发生了什么？

Status、Diff、Commit 在记录什么

Git 最实用的价值，不是上传 GitHub，而是把“修改过程”变得可以检查。

Status：现在有哪些变化还没处理

`git status` 会告诉你：哪些文件被修改、哪些是新文件、哪些变化准备进入下一次提交。它像项目现场的状态清单。

Diff: 前后到底差在哪

Diff 是差异对比。通常红色减号代表原内容被删除，绿色加号代表新内容被加入。Agent 说“只调整了排版”，但 Diff 显示正文也被重写，你就知道它的总结并不完整。

修改前

- 这是一份面向普通人的 GitHub 课程。

修改后

+ 欢迎来到改变世界的极客代码宇宙！

Commit: 确认这批修改值得留下

Commit 可以理解成一次写了说明的正式存档。每个 Commit 都带有作者、时间和说明，帮助以后知道当时为什么改。

它不是“每保存一下文件就自动产生”。更合理的节奏是：完成一件相对完整的小事，检查差异，确认没问题，再提交。



对普通人来说，不需要学会手动阅读复杂代码 Diff，但至少可以要求 Agent 给出“文件级摘要”，再对重要文件展示具体差异。

可以直接发给 Agent

先不要提交。请检查 Git 状态，列出本次修改涉及的所有文件；分别说明每个文件改了什么，有没有超出我的要求。对正文、配置和删除操作展示 Diff，等我确认后再创建 Commit。

最实用的三连问

改了哪些文件？具体改了什么？这批改动现在值得正式存档吗？

Branch 和 Pull Request 为什么有用

Branch：另开一条试验路线

Branch 就是分支。Git 仓库通常有一条主要分支，常见名字是 main。你可以从它出发，新建另一条路线，让 Agent 在里面尝试修改。

比如你的课程网页已经能正常使用，但想尝试全新的视觉风格。直接在主分支大改，失败后容易混乱；在新分支实验，就像从当前存档另开一条支线。

main 主分支

当前确认可用的版本。应该尽量保持清楚、稳定。

design-refresh 分支

Agent 尝试新版视觉。成功再合入，失败可以放弃。

Pull Request：一张“请检查这批修改”的申请单

Pull Request 通常简称 PR。它提出把一个分支的变化合并进另一个分支，并集中展示说明、Commit、检查结果和 Diff。

很多人以为 PR 只适合团队。其实即使项目只有你和 Agent，它也有价值：Agent 在分支里工作，最后形成一份可审查的修改包，你再决定要不要接收。

云端编码 Agent 更经常使用这种方式：它接到任务后创建分支、完成修改，再提交一个 PR 给人检查，而不是直接悄悄改掉主版本。

分支不是自动保险

如果 Agent 仍然在错误分支工作，或者你没有检查就直接合并，风险仍然存在。分支提供隔离，PR 提供审查机会，最后的判断依然属于你。

Commit 是存档点，**Branch** 是另一条路线，**Pull Request** 是把这条路线带回主线前的验收单。

08

这一章只回答：电脑里的项目怎样与 GitHub 保持联系？

Push、Pull 与本地、云端同步

Clone 以后，你的电脑里有一份本地仓库，GitHub 上有一份远程仓库。两边有关联，但不会在每次保存文件时自动同步。

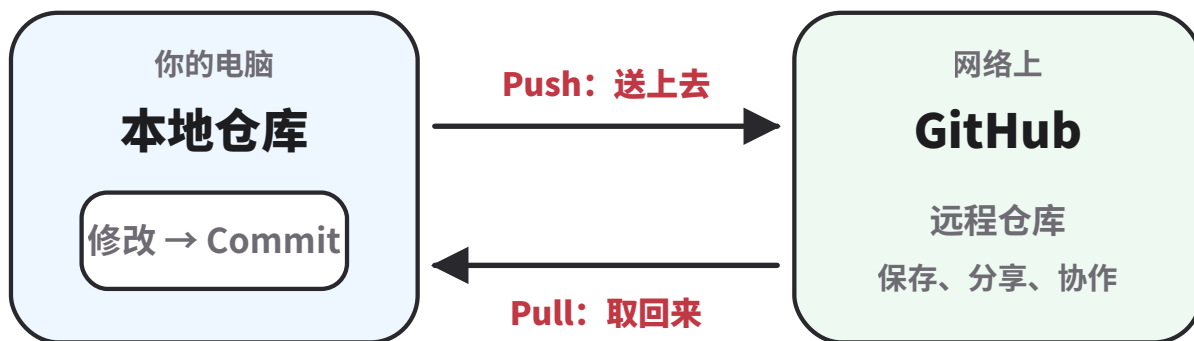
Push：把本地提交推上去

先在本地完成并 Commit，再把这些提交发送到 GitHub。没有 Commit 的普通修改不会因为 Push 自动变成远程版本。

Pull：把远程新变化拉下来

获取 GitHub 上的新提交，并更新当前本地分支。多人或多个 Agent 同时工作时尤其常见。

本地和云端像两个站点：Push 往右，Pull 往左



两边有关联，但不是每保存一次就自动同步

你可以把 GitHub 理解成“远程备份”，但它不是普通实时云盘。Git 更强调一批一批有说明的版本，而不是每改一个字就立即上传。

为什么会出现冲突？

如果本地和远程都修改了同一部分内容，Git 可能无法自动判断谁应该覆盖谁，于是产生 Merge Conflict，也就是合并冲突。

冲突不代表项目坏了，只是 Git 拒绝替你擅自做决定。Agent 可以帮助分析，但涉及重要正文、配置和数据时，仍应让它解释准备保留哪一边。

不要把“强制同步”当万能修复

强制 Push、硬重置或直接覆盖可能让另一边的历史消失。遇到同步问题，先确认本地和远程各自有什么，再决定保留哪份。

一句话带走

Commit 负责形成版本，Push 把版本送上去，Pull 把远程版本取回来。

PART 03 · 用得稳

让 GitHub 真正成为 Agent 的安全底座

09

这一章只回答：为什么项目规则最好跟项目放在一起？

给 Agent 留一份项目说明书

每次打开新对话都重新解释一遍项目，很浪费，也容易漏。现在不少 Agent 支持在仓库中放置专门的说明文件。

CODEx

AGENTS.md

Codex 会从项目根目录及相关目录读取指令，用来了解项目规则、检查方式和“不要做什么”。

CLAUDE CODE

CLAUDE.md

Claude Code 使用它保存项目级的长期说明，也可以在不同目录设置更具体的规则。

不同 Agent 支持的文件名和规则并不完全一样，但原理相同：把重复出现、长期有效的项目背景写进仓库，让下一次工作自动获得上下文。

普通人的项目说明里可以写什么

- 这个项目的用途，以及最重要的成品文件在哪里；
- 哪些目录可以修改，哪些原始资料不能动；
- 修改 HTML 后要检查手机和桌面显示；
- 大改前先建立分支，提交前必须展示 Diff；
- 禁止上传 API Key、私人照片和本地配置；
- 完成任务后怎样判断“真的完成”。

不要把说明文件写成一篇空泛长文。“认真检查”“保持高质量”很难执行；“修改完成后检查所有目录链接，不得删除正文段落”更有用。

可以直接发给 Agent

请根据这个项目当前结构，为我起草一份简短的项目说明文件。只保留长期有效的信息：项目用途、重要文件、允许修改的范围、完成后的检查方式、安全限制。不要写你可以从文件中自动看出来的冗长目录清单。

一句话带走

README 更偏向给人介绍项目；AGENTS.md、CLAUDE.md 等文件用来告诉 Agent 应该怎样在项目里工作。

一套普通人可用的 Agent workflow

理解前面的概念以后，不需要每次临时想怎么做。把下面这套顺序交给 Agent，已经能覆盖很多个人项目。

1 先确认位置

当前目录是不是正确项目？是不是 Git 仓库？现在在哪个分支？避免 Agent 在错误文件夹动手。

2 先看当前状态

有没有别人留下或上一次还没提交的修改？不要把不同任务混进同一次操作。

3 为不确定任务开分支

小改不必形式主义；但整体改版、升级依赖、批量删除等任务值得隔离。

4 让 Agent 修改并自己检查

要求它运行适合项目的测试、预览或链接检查，而不是改完就宣布完成。

5 看摘要和 Diff

确认改动范围没有超出要求，尤其关注删除、配置、依赖和数据文件。

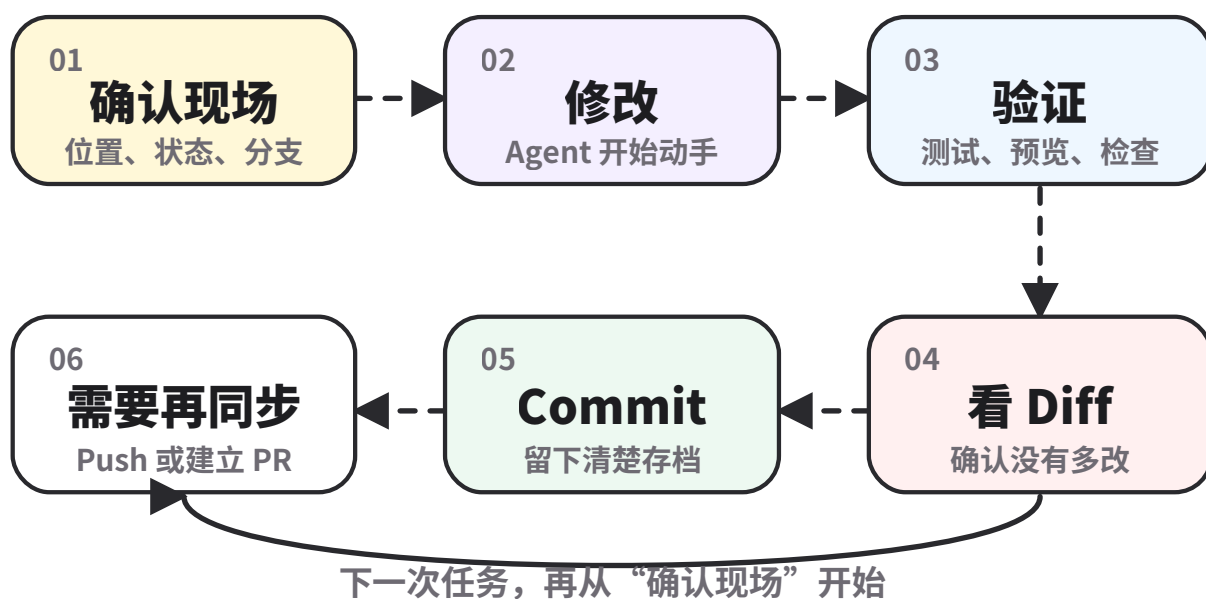
6 再 Commit

用一句能说明目的的话建立存档。不要把几件无关的修改塞进同一次 Commit。

7 需要时再 Push 或建 PR

本地验证过，再同步到 GitHub；重要修改通过 PR 留下更清楚的审查过程。

把七步压缩成一个不容易翻车的小闭环



这套流程看起来比“直接改”多几步，但这些步骤可以由 Agent 自己完成。你只需要在关键节点确认：工作位置、修改范围、验证结果和最终存档。

最值得养成的习惯不是“我会 Git”，而是：
Agent 动手前先看状态，动手后再看差异。

可以直接发给 Agent

请按这个顺序处理任务：确认仓库和分支 → 检查未提交修改 → 说明计划 → 完成修改 → 运行可用的检查 → 汇总所有改动并展示重要 Diff。先不要 Commit、Push 或合并，等我确认。

最小闭环

确认现场 → 修改 → 验证 → 看差异 → 存档 → 需要时同步。

这一章只回答：哪些东西绝对不能稀里糊涂上传？

公开仓库、API Key 与安全边界

Public 和 Private 不是装饰选项

公开仓库意味着互联网上任何人都可以看到内容；私有仓库只允许你和获得权限的人访问。第一次上传个人项目，如果不确定是否适合公开，先选 Private 更稳妥。

把私有仓库改成公开仓库前，要重新检查文件、提交历史、自动化日志和说明，因为公开的不只是一张首页。

.gitignore 是门口名单，不是历史橡皮擦

.gitignore 用来告诉 Git 哪些文件和目录不应进入新的提交，例如缓存、临时文件、本地配置和含密钥的 .env。

但如果某个文件已经被 Git 跟踪，仅仅后来写进 .gitignore 并不会自动把它从历史中抹掉。

API Key 泄露后，删除文件还不够

GitHub 的 Secret Scanning 会检查 Git 历史中的已知凭据。真实密钥一旦提交过，即使当前页面已经看不见，它仍可能存在于历史里。

正确思路通常是先轮换或撤销密钥，再处理仓库中的敏感数据，而不是只删除当前文件后假装没发生。

不要把 Token 当成“发给 Agent 的登录码”

Token 的作用接近密码。优先使用浏览器授权、系统凭据管理器或受支持的 Secrets 功能，不要在普通聊天里复制真实 Token，也不要把它硬编码进项目文件。

陌生仓库的命令也需要边界

安装命令可能下载程序、修改环境变量、安装依赖甚至执行脚本。遇到不熟悉的项目，可以让 Agent 先做静态检查和解释，再决定是否运行；不要因为 README 写着“一键安装”就自动获得信任。

- 先说明命令会访问哪些网站、写入哪些目录；
- 默认不要使用管理员权限；
- 不要读取与任务无关的私人文件；
- 涉及删除、覆盖、强制 Push、重写历史时必须再次确认；
- 尽可能先在独立目录、虚拟环境或测试分支中尝试。

安全底线

不确定就先私有；密钥不进仓库；危险命令先解释影响，再决定执行。

12

这一章只回答：真遇到事情时，普通人应该怎样开口？

五个常见场景，应该怎样让 Agent 做

场景一：我想安装 GitHub 上的一个工具

可以直接发给 Agent

先不要安装。确认仓库是否为官方来源，阅读 README、Releases、License、最近更新和主要 Issues。优先判断有没有适合普通用户的正式安装包；如果必须执行命令，先逐项解释会下载、安装和修改什么。

场景二：我有一个普通文件夹，想让 Agent 长期修改

可以直接发给 Agent

先检查这个文件夹是否适合初始化为 Git 仓库。列出不应跟踪的缓存、密钥和私人文件，起草 `.gitignore`；确认排除范围后再初始化，并为当前正常版本建立第一次 Commit。

场景三：Agent 要对 HTML 做整体改版

可以直接发给 Agent

先检查当前 Git 状态。如果工作区干净，为这次改版建立独立分支。保留正文内容，只修改视觉和排版；完成后检查目录链接、手机宽度和桌面显示，展示主要 Diff，不要直接合并到 `main`。

场景四：Agent 改坏了，我想恢复

可以直接发给 Agent

先不要执行任何恢复或重置命令。说明哪些修改已经 Commit、哪些还没有；列出可恢复的版本和每种方案会丢失什么。我的目标是保留有价值的内容并回到最近一个正常状态，等我确认方案后再执行。

场景五：我想把项目上传到 GitHub

可以直接发给 Agent

上传前先检查敏感信息、私人文件、超大文件和 `.gitignore`。默认创建 Private 仓库；说明当前本地分支、远程地址和准备 Push 的 Commit。不要上传未提交修改，也不要自动改成公开仓库。

你会发现，这些指令没有要求你记住某条 Git 命令。真正重要的是：你能说清目标、要求先检查、知道哪里需要确认，并且给 Agent 留出安全退路。

普通人使用 GitHub 的上限，不取决于会背多少命令，而取决于能不能在关键步骤提出正确要求。

课程终点

看到 GitHub 不慌，听懂 Agent 的基本动作，并能用自然语言要求它安全地完成工作。

最后，记住这 10 句话就够了

如果过几天忘了细节，只回来翻这一页。

- 1 Git 是本地版本记录，GitHub 是云端项目平台。
- 2 Repository 是文件、文件夹和修改历史组成的项目单位。
- 3 README 讲用途，Releases 放成品，Issues 记录问题，License 说明使用权利。
- 4 Star 是关注度参考，不是安全证书。
- 5 ZIP 是当前文件快照，Clone 是带历史和远程关系的完整副本。
- 6 Commit 是正式存档，Branch 是试验路线，Pull Request 是合并前的验收单。
- 7 Status 看现场，Diff 看具体变化，Push 和 Pull 负责本地与远程同步。
- 8 AGENTS.md、CLAUDE.md 等文件可以保存 Agent 的长期项目规则。
- 9 .gitignore 不能自动抹掉已经提交过的秘密；密钥泄露后应先轮换或撤销。
- 10 让 Agent 动手前先检查，动手后看差异，危险操作永远先解释影响。

资料来源、版本与边界

本课程不是对某一门现成视频课的压缩，而是依据 GitHub、Git 与主流 Agent 的官方文档，为普通人重新组织的原创轻量课程。定义和当前工具行为以官方资料为准；“网盘、游戏存档、试验路线”等是帮助理解的通俗比喻。

GitHub 与 Git 基础

- [GitHub Docs: Hello World](#)
- [GitHub Docs: About Git](#)
- [GitHub Docs: About repositories](#)
- [GitHub Docs: Getting started with Git](#)
- [GitHub Docs: Cloning a repository](#)
- [GitHub Docs: About READMEs](#)
- [GitHub Docs: About releases](#)
- [GitHub Docs: About issues](#)

身份、安全与许可

- [GitHub Docs: About authentication to GitHub](#)
- [GitHub Docs: Ignoring files](#)
- [GitHub Docs: Secret scanning](#)
- [GitHub Docs: Setting repository visibility](#)
- [GitHub Docs: Licensing a repository](#)

Agent 项目说明

- [Codex: Custom instructions with AGENTS.md](#)
- [Codex: Best practices](#)
- [Claude Code: How Claude remembers your project](#)

访问状态：上述文档均可公开阅读；若要实际创建、同步或修改个人 GitHub 仓库，需要注册并登录 GitHub。

更新边界：Git、仓库、Commit、Branch 等基础概念相对稳定；Agent 的说明文件、界面和登录方式可能变化，相关内容已按 2026-08-15 官方文档核查。